

AI FitTrack Project Planning

Phase 4

This plan organizes the completed backend work into dependency-based gates without inventing dates, team assignments or responsibilities that were not supplied.

Field	Value
Project	AI FitTrack
Submission	SmartBridge Naan Mudhalvan
Project scope	Backend-only REST API
Technology	Node.js, Express.js, MongoDB, Mongoose, JWT and Google Gemini AI
Source baseline	Latest uploaded backend ZIP and supplied verification evidence
Document status	Submission-ready content; student metadata and final video links remain pending

Prepared for academic evaluation. Secrets, credentials, private environment files and unverified claims are excluded.

Document contents

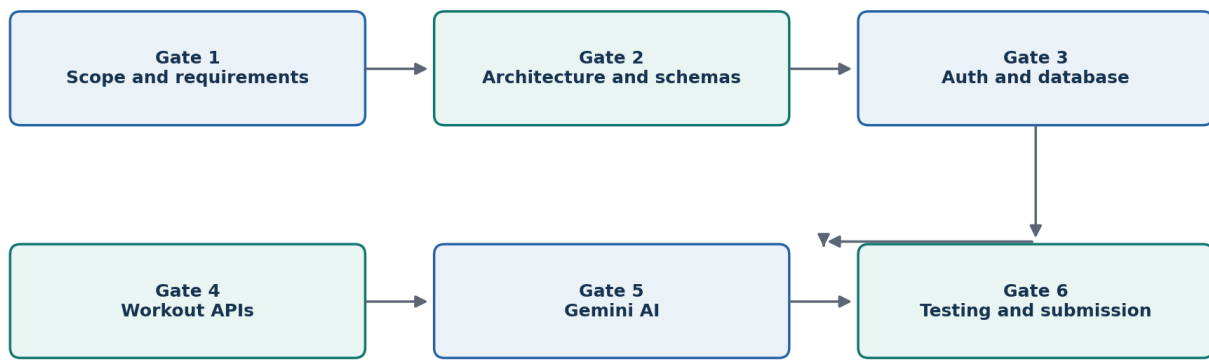
1. Planning approach
2. Work breakdown
3. Deliverables
4. Dependencies
5. Quality and risk plan
6. Roles and submission gates

Planning approach

Because the source package does not contain an approved calendar or team allocation, this plan uses ordered completion gates. Each gate has a measurable output and acceptance condition. Dates and assignees must be added only after the student or team confirms them.

Evidence-based project plan

Sequence uses dependency gates instead of unverified calendar dates



Actual dates and assignees are intentionally left for the student to confirm

Figure 1 Dependency-based project gates without unverified dates

Work breakdown structure

Gate	Work package	Deliverable	Acceptance evidence
1	Scope and requirements	Problem statement, objectives, actor and requirement list	Official document mapped to 24 backend requirements

Gate	Work package	Deliverable	Acceptance evidence
2	Architecture and data	Layer design, API design, User and Workout schemas	Source folders, schema constraints and diagrams agree
3	Authentication and database	MongoDB connection, User model, bcrypt, JWT, profile	Registration, login, profile and unauthorized evidence
4	Workout module	CRUD, search, validation and ownership	Eight live workout records plus automated tests
5	Gemini AI module	Recommendation, submitted insights and database insights	Three live sanitized Gemini response records
6	Verification and submission	Tests, documentation, repository and demonstration	95 automated passes recorded; final links still pending

Planned deliverables

- Backend source with dependency lockfile and safe .env.example.
- Postman collection containing health, authentication, workout and AI requests.
- Automated tests for authentication, workouts, validation, security and Gemini retry behavior.
- Sanitized live verification records with no passwords, tokens, database credentials or API keys.
- Eight-phase DOCX and PDF documentation set.
- Public GitHub repository and student-recorded demonstration video after final upload.

Dependency plan

Dependency	Needed by	Control
Node.js 20+ and npm	All development and tests	Use engines declaration and package-lock.json.
MongoDB	Auth and workout persistence	Connect before server listen; fail with actionable error.
JWT secret	Registration, login and protected routes	Fail at startup if absent; recommend at least 32 characters.
Gemini API key	AI recommendation and insights	Return safe 503 when absent; do not expose the key.

Dependency	Needed by	Control
Network access	Atlas and Gemini live checks	Use local mocked tests when external services are unavailable.
Postman or HTTP client	Demonstration and manual verification	Run the collection in the documented order.

Risk register

Risk	Likelihood	Impact	Implemented or planned response
Credential exposure	Medium	High	Commit only .env.example; hide terminal and Postman environment values during recording.
Atlas connection failure	Medium	High	Configurable DNS, IPv4 option, timeout and clear startup failure.
Gemini quota or downtime	Medium	Medium	Bounded retries, timeout, rate limit and safe normalized errors.
Cross-user data access	Low	High	JWT user reload and owner-scoped queries; automated isolation tests.
Invalid client input	Medium	Medium	Controller validation plus Mongoose validation and standard 400 responses.
Evidence mismatch after code change	Medium	Medium	Re-run Postman evidence after the final ZIP is frozen.
Missing submission identity or URLs	High	Medium	Complete the pending metadata checklist before upload.

Quality plan

- Review requirement-to-source traceability before declaring completion.
- Keep password, token and API-key values out of evidence and documentation.

- Run automated tests, smoke checks, Atlas checks and live Gemini checks on the frozen release.
- Verify response envelopes and status codes in Postman.
- Inspect DOCX and PDF output for clipping, unreadable tables and missing figures.
- Confirm the public repository excludes node_modules and every private .env file.

Role allocation template

Work area	Suggested responsibility	Confirmed assignee
Requirements and documentation	Maintain traceability and final report	Not supplied
Backend and database	Maintain Express, Mongoose and MongoDB behavior	Not supplied
Authentication and security	Review JWT, bcrypt, CORS and isolation	Not supplied
Gemini integration	Validate prompts, schema, retries and safety	Not supplied
Testing and demonstration	Run evidence flow and record video	Not supplied

Items requiring student confirmation

Pending item	What to provide
Student identity	Name, NM ID, register or roll number and department were not present in the supplied project files.
Institution details	College name, academic year, guide or mentor name and official submission date need confirmation.
Team details	Team name, member list and responsibility allocation were not supplied; no ownership is assigned in these documents.
Repository	Insert the final public GitHub repository URL after upload.
Deployment	Insert the deployed API base URL if the evaluator requires a hosted demonstration.
Demonstration video	Record the video using Phase 8 and insert the final shareable video URL.